

2026 《人工智能导论》大作业

任务名称： 可解释的谣言检测

完成组号： 10

小组人员： 张捷平、金思宇、唐苏睿

完成时间： 2026 年 6 月 7 日

分工情况：

唐苏睿：负责模型框架的构建和提供基础代码，以及部分报告编写。

金思宇：提供可尝试的其他算法，编写部分报告。

张捷平：对初代模型做改进、代码完善和调试，完善 `readme` 和报告。

注：最终模型即 `main` 下的代码，其余分支为构建模型时的参考方案。

1. 任务目标

本次大作业是面向英文社交媒体推文的谣言检测任务，需要基于谣言检测数据集，构建一个检测模型，该模型能够对数据集中的推文进行谣言检测，输出二分类结果与相应判断依据同时还需要具有一定的泛化能力，并且确保运行时间合理。

我们小组将以 `val.csv` 上的准确率和可解释性为核心目标，构建“Transformer 主分类 + 检索证据 + 规则纠偏 + 解释生成”的复合系统，在保证可复现的前提下尽量提升检测效果。

2. 具体内容

（1）实施方案

就如任务目标中所说的，实施方案主要就是构建“Transformer 主分类 + 检索证据 + 规则纠偏 + 解释生成”的复合系统。

首先是采用 BERTweet 作为主分类器，在给定的数据集上对 `tweet` 进行二分类微调，支持灵活的环境配置，可以直接在纯 `cpu` 上运行。

其次，系统并非是盲猜，而是会从训练集中寻找相似推文作为辅助决策证据，并通过 RRF 进行混合排序。

然后是规则纠偏。对于很容易误判的高风险样本，系统会结合证据一致性、官方通知模式、安全通知模式等进行纠偏。

最后在输出最终标签的同时，系统会综合当前推文特征、相似样本的标签分布以及事件语境，输出中文判断依据。

（2）核心代码分析

① `pipeline.py` 中的代码是整个系统的实现的主流程，其中的 `fuse_prediction` 函数用证据和投票结果来判断是否应该覆盖或调整初始预测，同时还可以结合语义分析加强判断

○○○

```
from collections import Counter

def fuse_prediction(base_prediction, evidence,
retrieval_cfg, text,
                    current_prediction_ref=None,
semantic_analysis=None):
    if not evidence:
        return base_prediction

    # 投票统计
    # weighted_votes, raw_votes = ...

    # 规则判断, 根据阈值和投票比例决定最终预测
    # if ...: return Prediction(...)
    # if ...: return Prediction(...)
    # if semantic_analysis ...: return
Prediction(...)

    return base_prediction
```

②dataset.py 负责数据预处理和数据格式统一。首先从 CSV 文件中读取原始数据, 并将不同数据集的字段名称映射为统一格式; 然后对推文文本进行标准化处理, 方便后续检索和相似度计算; 最后根据配置生成模型输入文本, 在需要时为文本添加事件名称前缀, 使模型能够利用事件背景信息进行分类, 从而为后续 Transformer 模型训练和检索增强模块提供规范化的数据输入。

○○○

```
@dataclass
class TweetRecord:
    tweet_id: str
    text: str
    label: int
    event: int
    normalized_text: str

def load_split(csv_path, config):
    # 读取CSV并统一字段名称
    pass

def add_normalized_text(df, retrieval_cfg):
    # 文本标准化处理
    pass

def add_model_input_text(df, raw_cfg):
    # 构造模型输入文本
    # 可选择添加事件前缀
    pass
```

③transformer.py 是系统里面的文本分类器, 负责 BERTweet 的训练、保存和推理。提供从文本到基础预测的完整流程, 为后续证据融合和决策模块提供基础判断。

```

○○○

import torch
from transformers import AutoTokenizer,
AutoModelForSequenceClassification
.....
def _ensure_loaded(self):
    # 加载模型和tokenizer
    pass

def fit(self, train_texts, train_labels,
val_texts=None, val_labels=None):
    # 构建DataLoader, 训练模型
    pass

def save(self):
    # 保存模型和tokenizer
    pass

def load(self):
    # 加载模型和tokenizer
    pass

@torch.inference_mode()
def predict_one(self, text):
    # 对单条文本预测label和置信度
    pass

```

④hybrid.py 用于推文的证据检索。它将 lexical 和 dense 两种检索方式封装, 根据配置可以单独使用或者混合使用。混合模式下 _hybrid_search 会将 lexical 和 dense 的结果排序, 为后续操作提供检索到的证据列表。

```

○○○

from dataclasses import dataclass
import pandas as pd
from rumor_system.retrieval.dense import
DenseRetriever
from rumor_system.retrieval.lexical import
LexicalRetriever, EvidenceItem

@dataclass
class HybridRetriever:
    train_df: pd.DataFrame
    retrieval_cfg: dict

    def __post_init__(self):
        # 初始化词汇检索和可选向量检索
        pass

    def search(self, text: str, top_k: int = 5) ->
list[EvidenceItem]:
        # 根据配置选择检索模式
        pass

    def _hybrid_search(self, text: str, top_k: int) -
> list[EvidenceItem]:
        # 词汇+向量混合检索并加权排序
        pass

```

(3) 检测结果分析

本项目最终采用“BERTweet 主分类器+检索证据增强+规则纠偏+判断依据生成”的稳定主线模型。在验证集 val.csv 上, 系统共正确分类 363 条样本, 总样本数为 401, 最终准确率达到 90.52%。

```
(C:\env\venv) PS D:\2025-2026-2\人工智能导论\Rumor_Classify-main> .\.venv\Scripts\python.exe scripts\evaluate.py --config configs\bertweet.yaml
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 11430.74it/s]
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 201/201 [00:00<00:00, 11170.44it/s]
[val] 20/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 40/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 60/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 80/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 100/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 120/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 140/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 160/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 180/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 200/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 220/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 240/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 260/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 280/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 300/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 320/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 340/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 360/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 380/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 400/401 processed | semantic_calls=0 online=0 offline=0 cache=0
[val] 401/401 processed | semantic_calls=0 online=0 offline=0 cache=0
Run output directory: outputs\2026-06-06-23-49-30
{'accuracy': 0.9052369077306733, 'total': 401, 'correct': 363}
```

图 1 实验测得最终准确率

为比较不同模块对性能的贡献，我们设计了一组递进式对比实验。首先从“仅依赖相似样本投票”的混合检索基线开始，用于评估纯证据检索方案的效果；随后加入 BERTweet 主分类器，考察预训练 Transformer 对 tweet 语义和表达风格建模带来的提升；再进一步加入证据冲突处理、官方更新识别、安全通知识别等规则纠偏模块，以验证复合式方案能否在高频错例上继续提分。除此之外，我们还尝试了事件前缀输入和语义分析改判两条扩展路线，分别用于检验事件域信息和更高层语义信息是否能带来额外增益。

实验结果表明,混合检索基线准确率为 84.29%,加入 BERTweet 后提升至 85.54%,说明 Transformer 的语义建模是必要的;继续加入规则纠偏后,主线模型提升到 90.52%,说明“主分类器 + 检索证据 + 定向纠偏”的复合结构更适合本任务。相比之下,事件前缀输入实验和语义分析实验都未带来进一步提升,说明并不是所有看起来合理的扩展方案都能在当前数据集上产生实际收益。

方案名称	正确样本数	总样本数	准确率	说明
混合检索基线	338	401	84.29%	仅依赖检索相似样本投票
主分类器与检索	343	401	85.54%	加入 BERTweet 主分类器
规则纠偏主线（早期）	358	401	89.28%	加入证据冲突与官方更新纠偏
事件前缀输入实验	349	401	87.03%	显式加入事件前缀，效果下降
语义分析实验	345	401	86.03%	引入语义分析改判，整体下降
最终主线模型	363	401	90.52%	加入谣言式传播与安全通知纠偏

表 1 各方案准确率对比表

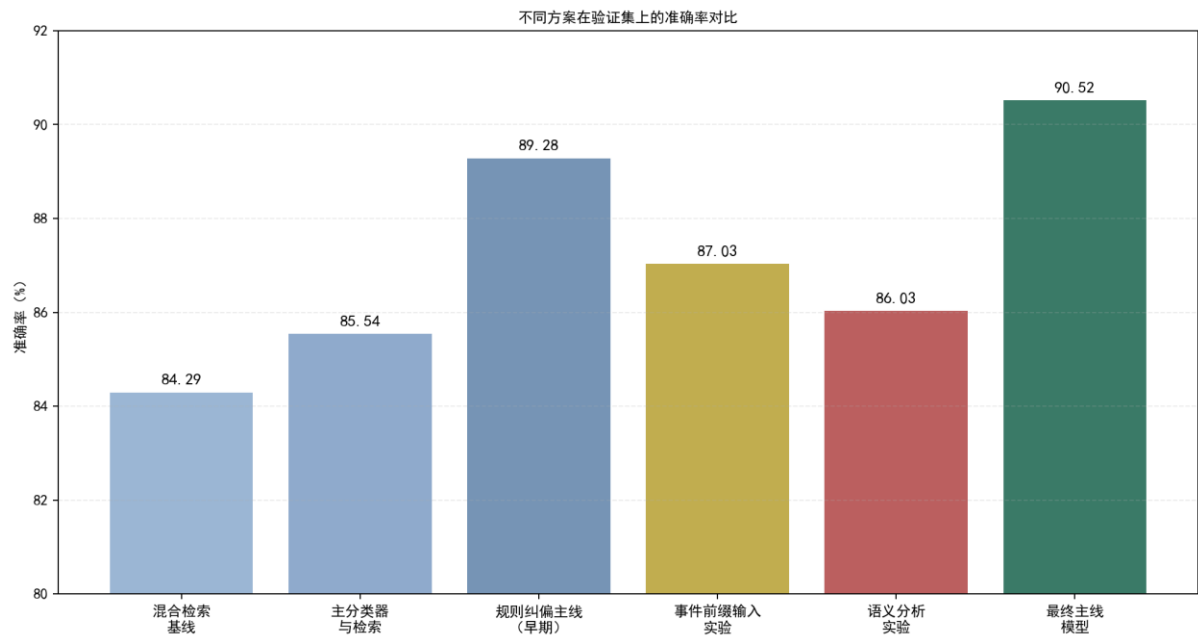


图 2 不同方案在验证集上的准确率对比

从事件维度来看，不同事件的识别难度差异明显。德国之翼空难和悉尼人质事件上的准确率相对较低，说明“突发快讯”“现场播报”“官方更新”等新闻化表达更容易干扰模型；而古利特事件、埃博拉传闻和多伦多王子相关样本表现更稳定。很明显，当前误差并不是随机分布，而是与具体事件的语言风格和传播模式密切相关。

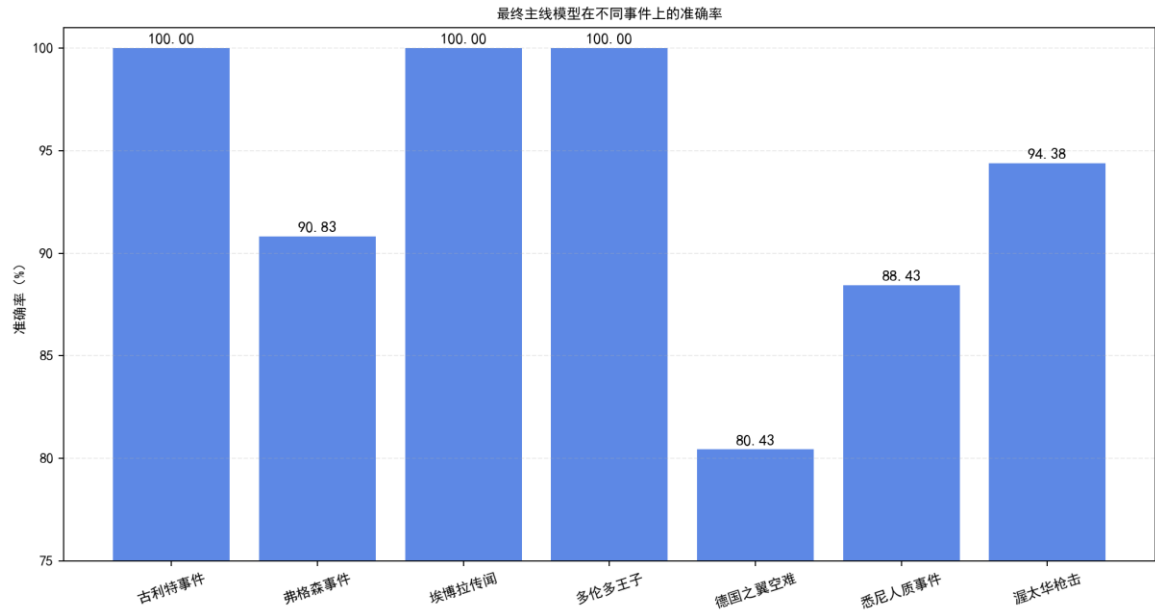


图 3 最终主线模型在不同事件上的准确率

（4）判断依据的分析

本项目设计了判断依据生成模块，以提高系统的可解释性。其核心思想是将“当前推文的关键表达”“检索到的相似样本及其标签分布”“当前样本在传播中的可能角色”整合起来，再生成与最终预测一致的说明。这样做能够避免出现结论正确但解释飘忽的问题，也更符合课程项目对可信解释的要求。

当前判断依据主要从三个层面展开。第一是文本层证据，即当前 tweet 自身的关键触发短语；第二是检索层证据，即相似训练样本整体更偏向谣言还是非谣言；第三是传播角色层证据，即系统判断推文。三部分共同决定了最终的判断依据结构。

从决策来源分布可以看出，大多数样本仍然由主分类器直接完成判断，但对于少量高风险样本，系统会进一步调用检索投票、规则纠偏等模块。所以可解释性模块既要解释主分类器的判断，也要解释为什么某些样本会被规则层重新修正。

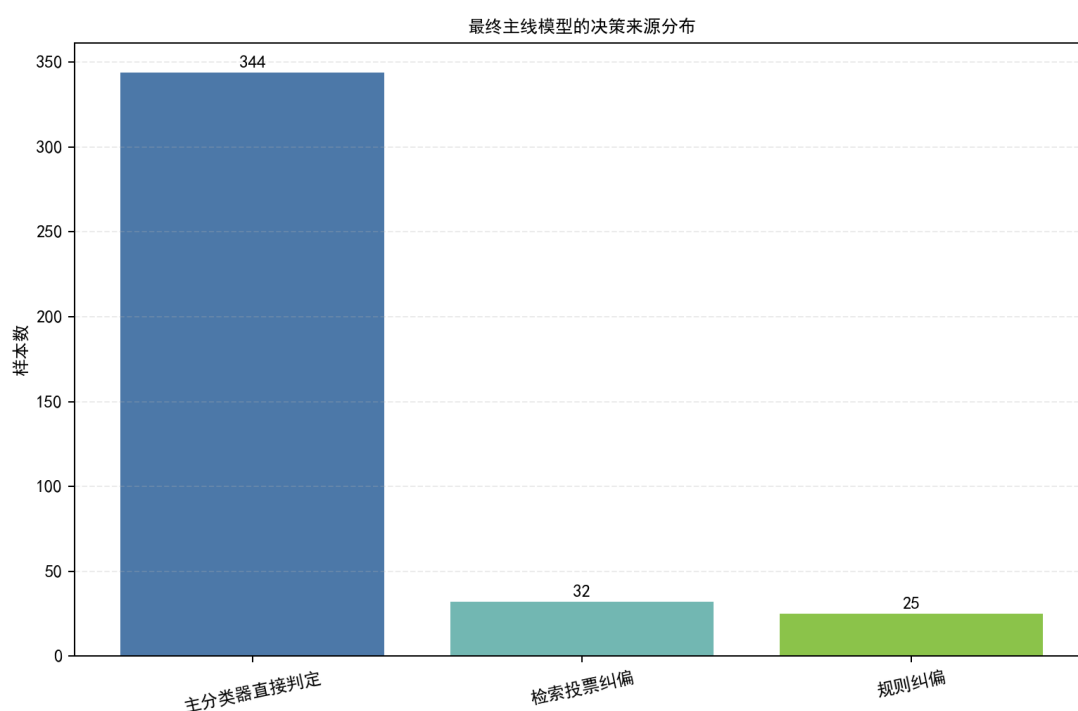


图 4 最终主线模型的决策来源分布

从示例上看，对于通知类文本，系统会倾向解释为机构性公告而非谣言传播；而对于连续强指控的文本，系统则会解释为更接近对未证实指控的谣言式扩散。

示例类型	输入文本片段	判断依据
通知类非谣言	UPDATE: #uOttawa courses + exams officially cancelled for the whole day. Lockdown still in effect. Stay indoors, stay safe #Ottawa #ottcity	该推文被判定为非谣言，置信度为 0.82，因为它在传播中更像一条机构安全通知。其中，“officially cancelled”，“lockdown still in effect”，“stay indoors”等表述是触发判断的重要文本信号。检索到的相似样本中包含 2 条谣言样本、1 条非谣言样本，整体证据更偏向谣言。最相近样本与当前文本在 ottawa, safe, stay, cancelled 等词汇上存在重合，且 top evidence 与最终标签不完全一致，因此系统还结合了整体证据分布和分类器置信度。
强指控式谣言传播	Shoot unarmed kid. Conceal evidence. Impose martial law. Harass reporters. Smear the victim. Worst. Police. Ever. #Ferguson #MikeBrown	该推文被判定为谣言，置信度为 0.95，因为它在传播中更像一条对未证实指控的谣言式传播。其中，“conceal evidence”，“smear the victim”，“martial law”等表述是触发判断的重要文本信号。检索到的相似样本中包含 1 条谣言样本、2 条非谣言样本，整体证据更偏向非谣言。最相近样本与当前文本在 ferguson, shoot, unarmed, ever 等词汇上存在重合，且 top evidence 与最终标签一致。

表 2 中文判断依据实例

3. 工作总结

（1）收获心得

通过本次课程大作业，我们对人工智能系统的构建过程有了更加全面和深入的认识。以社交媒体谣言检测与解释为研究对象，从最初的数据预处理、模型训练，到后续的检索增强、结果融合以及大模型解释生成，完整经历了一个人工智能应用系统从设计到实现的全过程。整个大作业完成的过程中，不仅巩固了课程的理论知识，也提升了实际工程开发能力，包括数据处理、模型训练、模块化编程等方面的能力。更重要的是，我们认识到人工智能项目的实现不仅需要算法知识，还需要考虑数据质量、系统架构设计、模型的解释性以及实际应用场景等多方面因素。

（2）遇到问题及解决思路

在项目实现过程中，我们首先遇到的问题是：仅依赖单条推文文本分类时，模型容易学习到表层伪相关特征，例如将“突发”“枪击”等高冲击词汇误当作谣言信号，而把一些看似新闻播报的谣言样本误判为非谣言。对此，我们先对错例进行分析，将误判归纳为新闻快讯误判、官方通知误判、强指控传播漏判等几类模式，再据此设计有针对性的纠偏规则。

第二个问题是主分类器在高风险样本上不够稳定，因此我们引入检索增强模块，从训练集中寻找相似样本作为辅助证据，并在证据与主模型冲突时结合官方更新、安全通知、谣言式传播等模式进行定向修正。我们还尝试过让大语言模型直接参与改判，但实验表明整体效果下降，因此最终仅保留其在解释层中的辅助

作用。

最后，为保证运行效率与复现性，我们保留了可在 CPU 上稳定运行的主线模型，并通过缓存和进度打印优化了实验过程。整体来看，本项目的效果提升依赖于错误分析、证据设计、规则融合和工程实现的共同优化。

4. 课程建议

人工智能导论课程内容覆盖广泛，为我们建立了较完整的人工智能基础知识体系。但在学习过程中感觉部分理论知识与实际应用之间仍有一定距离。建议在课程教学中增加更多实践性内容，在期末大作业之外，可以安排一些与课堂内容密切相关的小任务，层层递进。

另外，对于课程的大作业方面，由于此前没怎么经历过人工智能项目的开发，大作业的整体工作量相对较大，学习成本比较高。建议之后可以在课程中提供更加详细的项目指导文档或者基础代码框架。